C# BASICS / PRACTICE TASK 02

Simple Banking System

A console application challenge covering type conversion, methods, ref/out parameters, and exception handling.

Beginner / Beginner+	C# Console Application	10 Marks + 2 Bonus

TASK 00 | Project overview (- Marks)

Build a simple banking app that lets a user check a balance, deposit and withdraw money, and view a transaction summary. The menu should repeat until the user exits.

Topics: Parse / TryParse / Convert; try / catch / finally / throw; multiple catches; methods and return values; ref and out.

Rules: Use separate methods; call them from Main(); use meaningful names; keep the app running after handled errors; do not create custom exception classes.

TASK 01 | Account setup & conversion (2 Marks)

- Ask for the user name and initial balance. Read input with Console.ReadLine().
- Validate and convert the balance using decimal.TryParse(). If invalid, ask again.
- Reject negative initial balances. Display a welcome message and starting balance.
- Demonstrate one additional built-in conversion technique (Parse or Convert) in an appropriate safe part of the program, and handle possible invalid input.

```
Enter your name: Ahmed
Enter initial balance: abc
Invalid balance. Please enter a valid number.
Enter initial balance: 1000
Account created successfully! Balance: 1000 EGP
```

TASK 02 | Methods & banking operations (2 Marks)

Create and call these methods:

- ShowMenu() - displays the options and returns the selected option as an int.
- Deposit(ref decimal balance, decimal amount) - updates the original balance.
- Withdraw(ref decimal balance, decimal amount) - updates the original balance.
- DisplayBalance(decimal balance) - displays the balance without modifying it.

```
========== BANK MENU ==========
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. View Transaction Summary
5. Exit
Choose an option:
```

TASK 03 | Using out parameters (1.5 Marks)

Implement bool TryReadAmount(string input, out decimal amount). It must return true only if the text converts to a positive decimal, otherwise false. Use this method for deposit and withdrawal inputs.

```
Enter deposit amount: hello
Invalid amount!
Enter deposit amount: 500
Deposit successful! Current Balance: 1500 EGP
```

TASK 04 | Exception handling (2 Marks)

- Use try / catch for transaction operations and multiple catch blocks when appropriate.
- Throw ArgumentOutOfRangeException for zero or negative transaction amounts.
- Throw InvalidOperationException when withdrawal exceeds the available balance.
- Include a finally block that prints "Transaction attempt completed."
- Use ex.Message when useful. A handled exception must not crash the app.
- TryParse validates text conversion; throw/catch handles business rule violations. Both must be demonstrated.

```
Current Balance: 1000 EGP
Enter withdrawal amount: 2000
Error: Insufficient balance.
Transaction attempt completed.
Returning to main menu...
```

TASK 05 | Transaction summary (1.5 Marks)

- Track successful deposit count, successful withdrawal count, total deposited and total withdrawn.
- Create DisplayTransactionSummary() to show these values and the current balance.
- Update statistics only after successful transactions.

```
====== TRANSACTION SUMMARY ======
Successful Deposits:    3
Successful Withdrawals: 2
Total Deposited: 1500 EGP
Total Withdrawn:  600 EGP
Current Balance: 1900 EGP
```

TASK 06 | Menu validation & flow (1 Marks)

- Repeat the menu until option 5 (Exit).
- Handle non-numeric and out-of-range menu choices with friendly messages.
- Keep Main() readable by organizing actions in methods.

BONUS CHALLENGE | Exception Filters (+2 Marks)

Use catch (...) when (...) to handle an InvalidOperationException caused by an insufficient balance. Print "Withdrawal rejected: Insufficient funds." Use another appropriate catch to handle other InvalidOperationException cases. No custom exception types.

Submission checklist

- Submit a GitHub repository link containing the complete C# Console Application.
- The code must compile and run; demonstrate invalid input, successful transactions, and insufficient funds.
- All required methods and both ref / out parameter usages must be visible in the implementation.

Scoring: Account setup 2 + Methods 2 + out 1.5 + Exceptions 2 + Summary 1.5 + Menu flow 1 = 10 marks. Bonus = 2 extra marks.